

CAPÍTULO 1

Introducción

1.1 ¿Qué son los algoritmos?

Un **algoritmo** es un procedimiento computacional bien definido que toma algún valor, o conjunto de valores, como **entrada** y produce algún valor, o conjunto de valores, como **salida** en un intervalo de tiempo finito.

También podemos ver un algoritmo como una herramienta para resolver un **problema computacional** bien especificado. El enunciado del problema especifica en términos generales la relación deseada entre la entrada y la salida. El algoritmo describe un procedimiento computacional específico para lograr esa relación.

Definición 1.1 (Algoritmo). *Un algoritmo es una secuencia finita de instrucciones precisas para realizar una tarea, que dado un estado inicial, terminará en un estado final definido.*

1.1.1 Ejemplo: El problema del ordenamiento

El **problema del ordenamiento** se define formalmente como:

Problema del Ordenamiento

Entrada: Una secuencia de n números $\langle a_1, a_2, \dots, a_n \rangle$.

Salida: Una permutación (reordenamiento) $\langle a'_1, a'_2, \dots, a'_n \rangle$ de la secuencia de entrada tal que $a'_1 \leq a'_2 \leq \dots \leq a'_n$.

Por ejemplo, dada la secuencia de entrada $\langle 31, 41, 59, 26, 41, 58 \rangle$, un algoritmo de ordenamiento devuelve como salida la secuencia $\langle 26, 31, 41, 41, 58, 59 \rangle$.

A tal secuencia de entrada se le llama una **instancia** del problema del ordenamiento. En general, una **instancia de un problema** consiste en la entrada necesaria para calcular una solución al problema.

1.1.2 ¿Qué tipo de problemas resuelven los algoritmos?

El ordenamiento no es el único problema computacional para el cual se han desarrollado algoritmos. Las aplicaciones prácticas de los algoritmos son ubicuas e incluyen:

- **Búsqueda en Internet:** Motores de búsqueda utilizan algoritmos para encontrar páginas relevantes entre miles de millones de documentos.
- **Redes sociales:** Algoritmos determinan qué contenido mostrar, sugieren amigos y detectan comunidades.
- **Criptografía:** Protege información sensible usando algoritmos de encriptación basados en teoría de números.
- **Asignación de recursos:** Desde rutas de vehículos hasta asignación de frecuencias en telecomunicaciones.

- **Bioinformática:** Comparación de secuencias de ADN, predicción de estructuras de proteínas.
- **Inteligencia Artificial:** Aprendizaje automático, procesamiento de lenguaje natural, visión por computadora.

1.2 Estructuras de datos

Una **estructura de datos** es una forma de almacenar y organizar datos para facilitar el acceso y las modificaciones. Ninguna estructura de datos única funciona bien para todos los propósitos, por lo que es importante conocer las fortalezas y limitaciones de varias de ellas.

Definición 1.2 (Estructura de Datos). *Una estructura de datos es una colección de valores, las relaciones entre ellos, y las funciones u operaciones que se pueden aplicar a los datos.*

1.2.1 Estructuras de datos fundamentales

Arreglos

Un **arreglo** es una colección de elementos del mismo tipo almacenados en posiciones contiguas de memoria. Cada elemento se puede acceder directamente mediante un índice.

- **Acceso:** $O(1)$ - tiempo constante
- **Búsqueda:** $O(n)$ - tiempo lineal (sin ordenar)
- **Inserción/Eliminación:** $O(n)$ - requiere desplazar elementos

Listas enlazadas

Una **lista enlazada** es una estructura donde cada elemento (nodo) contiene un dato y una referencia al siguiente elemento.

- **Acceso:** $O(n)$ - debe recorrer desde el inicio
- **Inserción/Eliminación:** $O(1)$ - solo cambia referencias (si ya está en posición)

Pilas y Colas

- **Pila (Stack):** Estructura LIFO (Last In, First Out). Operaciones: push, pop.
- **Cola (Queue):** Estructura FIFO (First In, First Out). Operaciones: enqueue, dequeue.

1.3 Análisis de algoritmos

Antes de poder analizar un algoritmo, debemos tener un modelo de la tecnología de implementación que usaremos, incluyendo un modelo para los recursos de esa tecnología y sus costos.

1.3.1 Modelo RAM

Asumimos un modelo genérico de máquina de acceso aleatorio (**RAM**) de un solo procesador, en el cual las instrucciones se ejecutan una tras otra, sin operaciones concurrentes.

El modelo RAM contiene instrucciones comúnmente encontradas en computadoras reales:

- Aritméticas: suma, resta, multiplicación, división, residuo, piso, techo
- Movimiento de datos: cargar, almacenar, copiar
- Control: salto condicional e incondicional, llamada y retorno de subrutinas

Cada una de estas instrucciones toma una cantidad **constante** de tiempo.

1.3.2 Tiempo de ejecución

El **tiempo de ejecución** de un algoritmo sobre una entrada particular es el número de operaciones primitivas o “pasos” ejecutados.

Es conveniente definir la noción de paso de manera que sea lo más independiente posible de la máquina. Por ahora, adoptamos la siguiente perspectiva: se requiere una cantidad constante de tiempo para ejecutar cada línea de nuestro pseudocódigo.

1.4 Notación asintótica

Cuando observamos tamaños de entrada lo suficientemente grandes para hacer relevante solo el orden de crecimiento del tiempo de ejecución, estudiamos la **eficiencia asintótica** de los algoritmos.

1.4.1 Notación O (O grande)

La notación O caracteriza un **límite superior** en el comportamiento asintótico de una función. Es decir, acota qué tan rápido crece una función.

Definición 1.3 (Notación O grande). Para una función dada $g(n)$, denotamos por $O(g(n))$ al conjunto de funciones:

$$O(g(n)) = \{f(n) : \text{existen constantes positivas } c \text{ y } n_0 \text{ tales que } 0 \leq f(n) \leq c \cdot g(n) \text{ para todo } n \geq n_0\}$$

Escribimos $f(n) = O(g(n))$ para indicar que $f(n)$ es miembro del conjunto $O(g(n))$.

1.4.2 Notación Ω (Omega grande)

La notación Ω proporciona un **límite inferior** asintótico.

Definición 1.4 (Notación Omega grande).

$$\Omega(g(n)) = \{f(n) : \text{existen constantes positivas } c \text{ y } n_0 \text{ tales que } 0 \leq c \cdot g(n) \leq f(n) \text{ para todo } n \geq n_0\}$$

1.4.3 Notación Θ (Theta)

La notación Θ acota una función desde arriba y desde abajo asintóticamente.

Definición 1.5 (Notación Theta).

$$\Theta(g(n)) = \{f(n) : \text{existen constantes positivas } c_1, c_2 \text{ y } n_0 \text{ tales que } 0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \text{ para todo } n \geq n_0\}$$

1.4.4 Complejidades comunes

1.5 Correctitud de algoritmos

Un algoritmo es **correcto** si, para cada instancia de entrada, termina con la salida correcta. Decimos que un algoritmo correcto **resuelve** el problema computacional dado.

1.5.1 Invariantes de ciclo

Una técnica común para demostrar la correctitud de un algoritmo es usar **invariantes de ciclo**. Debemos demostrar tres cosas sobre un invariante de ciclo:

1. **Inicialización:** Es verdadero antes de la primera iteración del ciclo.
2. **Mantenimiento:** Si es verdadero antes de una iteración del ciclo, permanece verdadero antes de la siguiente iteración.
3. **Terminación:** Cuando el ciclo termina, el invariante nos da una propiedad útil que ayuda a mostrar que el algoritmo es correcto.

Notación	Nombre	Ejemplo
$O(1)$	Constante	Acceso a arreglo por índice
$O(\log n)$	Logarítmica	Búsqueda binaria
$O(n)$	Lineal	Búsqueda lineal
$O(n \log n)$	Linealítmica	Merge Sort, Quick Sort (promedio)
$O(n^2)$	Cuadrática	Bubble Sort, Insertion Sort
$O(n^3)$	Cúbica	Multiplicación de matrices (ingenua)
$O(2^n)$	Exponencial	Subconjuntos de un conjunto
$O(n!)$	Factorial	Permutaciones

Cuadro 1.1: Complejidades computacionales comunes

1.6 Diseño de algoritmos

Existen varios paradigmas para diseñar algoritmos:

1.6.1 Fuerza bruta

Examina todas las posibles soluciones y selecciona la mejor. Simple pero generalmente ineficiente.

Complejidad típica: Exponencial o factorial.

1.6.2 Divide y vencerás

Divide el problema en subproblemas más pequeños del mismo tipo, resuelve los subproblemas recursivamente y combina las soluciones.

Ejemplos: Merge Sort, Quick Sort, Búsqueda Binaria.

Recurrencia típica: $T(n) = aT(n/b) + f(n)$

1.6.3 Programación dinámica

Resuelve subproblemas superpuestos y almacena sus resultados para evitar recalcularlos.

Características:

- **Subestructura óptima:** La solución óptima contiene soluciones óptimas de subproblemas.
- **Subproblemas superpuestos:** Los mismos subproblemas se resuelven múltiples veces.

Ejemplos: Fibonacci, Knapsack, Camino más corto.

1.6.4 Algoritmos voraces (Greedy)

Construye una solución paso a paso, eligiendo en cada paso la opción localmente óptima.

Ejemplos: Algoritmo de Dijkstra, Árbol de expansión mínima (Prim, Kruskal).

1.6.5 Backtracking

Construye soluciones incrementalmente, abandonando candidatos (“retrocediendo”) tan pronto como determina que no pueden llevar a una solución válida.

Características:

- Explora el espacio de soluciones sistemáticamente.
- “Poda” ramas que no pueden llevar a solución.
- Útil para problemas de satisfacción de restricciones.

Ejemplos: N-Reinas, Sudoku, Laberintos.

```

1 def backtrack(solucion_parcial):
2     if es_solucion(solucion_parcial):
3         procesar_solucion(solucion_parcial)
4         return
5
6     for candidato in generar_candidatos(solucion_parcial):
7         if es_valido(candidato, solucion_parcial):
8             solucion_parcial.append(candidato)
9             backtrack(solucion_parcial)
10            solucion_parcial.pop() # Retroceder

```

1.6.6 Búsqueda local

Comienza con una solución inicial y la mejora iterativamente mediante modificaciones locales.

Algoritmos:

- **Hill Climbing:** Siempre se mueve hacia vecinos mejores.
- **Simulated Annealing:** Permite movimientos a soluciones peores con cierta probabilidad.
- **Algoritmos genéticos:** Evoluciona una población de soluciones.

1.7 Clases de Complejidad

1.7.1 Algoritmos factibles y no factibles

Definición 1.6 (Algoritmo de tiempo polinomial). *Un algoritmo es de **tiempo polinomial** si su complejidad temporal es $O(n^k)$ para alguna constante k .*

Los algoritmos de tiempo polinomial se consideran **factibles** o **eficientes**. Los algoritmos exponenciales ($O(2^n)$, $O(n!)$) son generalmente **no factibles** para entradas grandes.

Complejidad	$n = 20$	$n = 50$	$n = 100$
$O(n)$	20	50	100
$O(n^2)$	400	2,500	10,000
$O(n^3)$	8,000	125,000	1,000,000
$O(2^n)$	1,048,576	10^{15}	10^{30}
$O(n!)$	10^{18}	10^{64}	10^{157}

Cuadro 1.2: Crecimiento de diferentes complejidades (número de operaciones).

1.7.2 La clase P

Definición 1.7 (Clase P). *P es el conjunto de problemas de decisión que pueden ser resueltos por un algoritmo determinista en tiempo polinomial.*

Ejemplos en P:

- Ordenamiento
- Búsqueda en grafos
- Camino más corto
- Flujo máximo

1.7.3 La clase NP

Definición 1.8 (Clase NP). *NP* (Nondeterministic Polynomial) es el conjunto de problemas de decisión cuyas soluciones pueden ser **verificadas** en tiempo polinomial.

Es decir, si alguien te da una supuesta solución, puedes verificar si es correcta en tiempo polinomial.

Ejemplos en NP:

- **SAT:** ¿Existe una asignación que satisfaga una fórmula booleana?
- **Clique:** ¿Existe un clique de tamaño k en un grafo?
- **Subset Sum:** ¿Existe un subconjunto que sume exactamente S ?

Nota: $P \subseteq NP$ (todo lo que se puede resolver rápido, se puede verificar rápido).

1.7.4 Problemas NP-Completo

Definición 1.9 (NP-Completo). *Un problema es NP-Completo si:*

1. *Está en NP.*
2. *Todo problema en NP se puede reducir a él en tiempo polinomial.*

Los problemas NP-Completo son los “más difíciles” de NP. Si encontramos un algoritmo polinomial para cualquier problema NP-Completo, entonces $P = NP$.

Problemas NP-Completo famosos:

- SAT (Satisfacibilidad booleana)
- 3-SAT
- Clique
- Vertex Cover
- Hamiltonian Path/Cycle
- Traveling Salesman Problem (TSP) - versión decisión
- Knapsack (0/1)
- Graph Coloring

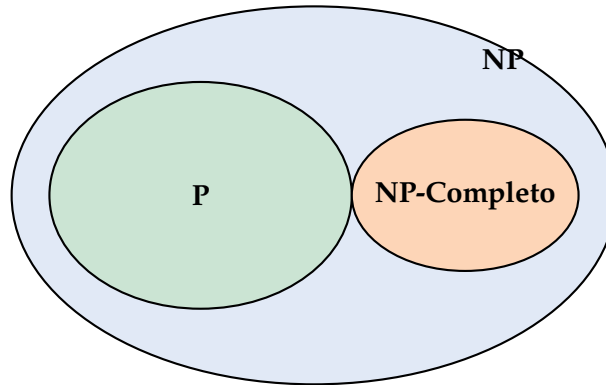
1.7.5 La pregunta P vs NP

La pregunta “¿Es $P = NP$?” es uno de los problemas abiertos más importantes en ciencias de la computación y matemáticas. Es uno de los **Problemas del Milenio** con un premio de \$1,000,000.

Si $P = NP$: Muchos problemas actualmente difíciles serían fáciles. La criptografía moderna colapsaría.

Si $P \neq NP$: Existen problemas inherentemente difíciles que nunca podrán resolverse eficientemente.

Consenso: La mayoría de los investigadores creen que $P \neq NP$.



(Asumiendo $P \neq NP$)

Figura 1.1: Relación entre las clases P, NP y NP-Completo.

1.7.6 Soluciones aproximadas

Cuando enfrentamos problemas NP-Completo, podemos usar:

1. **Algoritmos de aproximación:** Garantizan soluciones dentro de un factor de la óptima.
2. **Heurísticas:** Sin garantías, pero funcionan bien en la práctica.
3. **Algoritmos aleatorizados:** Usan aleatoriedad para encontrar buenas soluciones.
4. **Casos especiales:** Algunos casos del problema pueden ser tratables.
5. **Algoritmos parametrizados:** Eficientes cuando algún parámetro es pequeño.

1.8 Elementos de Programación

1.8.1 Tipos de datos fundamentales

Los tipos de datos primitivos son los bloques básicos de construcción:

Tipo	Python	Tamaño típico	Ejemplo
Entero	int	Variable	42, -17
Real	float	64 bits	3.14159
Booleano	bool	1 bit (lógico)	True, False
Caracter	str (longitud 1)	Variable	'a', 'Z'
Cadena	str	Variable	"Hola"

Cuadro 1.3: Tipos de datos primitivos.

1.8.2 Estructuras de control

Secuencia: Instrucciones ejecutadas en orden.

Selección (condicionales):

```

1 if condicion:
2     # bloque si verdadero
3 elif otra_condicion:
```

```

4     # bloque alternativo
5 else:
6     # bloque por defecto

```

Iteración (ciclos):

```

1 # Ciclo for - iteracion definida
2 for i in range(n):
3     # se ejecuta n veces
4
5 # Ciclo while - iteracion indefinida
6 while condicion:
7     # mientras condicion sea verdadera

```

1.8.3 Manejo de memoria

En Python, la memoria es manejada automáticamente mediante:

- **Asignación dinámica:** Los objetos se crean en el heap.
- **Recolección de basura:** Objetos no referenciados son liberados automáticamente.
- **Conteo de referencias:** Python cuenta cuántas referencias apuntan a cada objeto.

Paso por referencia vs. valor:

```

1 # En Python, todo es por referencia al objeto
2 # Pero los inmutables (int, str, tuple) no pueden modificarse
3
4 def modificar(lista, numero):
5     lista.append(4)    # Modifica la lista original
6     numero = numero + 1 # Crea nuevo objeto, no modifica original
7
8 mi_lista = [1, 2, 3]
9 mi_numero = 10
10 modificar(mi_lista, mi_numero)
11 print(mi_lista)    # [1, 2, 3, 4] - modificada
12 print(mi_numero)   # 10 - sin cambios

```

1.9 Ejercicios

1. Ordena las siguientes funciones por orden de crecimiento: n , $\sqrt{n}$, $n^{1.5}$, n^2 , $n \log n$, $n \log \log n$, $n \log^2 n$, $n \log(n^2)$, 2^n , $2^{n/2}$, $n!$.
2. Demuestra que $n^2 + 100n + 1000 = O(n^2)$.
3. ¿Es $2^{n+1} = O(2^n)$? ¿Es $2^{2n} = O(2^n)$?
4. Para cada función $f(n)$ y tiempo t en la siguiente tabla, determina el tamaño máximo n de un problema que puede resolverse en tiempo t , asumiendo que el algoritmo tarda $f(n)$ microsegundos.
5. Describe un algoritmo que, dado un conjunto S de n enteros y otro entero x , determine si existen dos elementos en S cuya suma sea exactamente x .

1.10 Referencias

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3ra ed.). MIT Press.
- Knuth, D. E. (1997). *The Art of Computer Programming*. Addison-Wesley.
- Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4ta ed.). Addison-Wesley.